

Introducción C#

Módulo 1

Introducción a .NET

¿Qué es .NET?



Es un amplio conjunto de bibliotecas de desarrollo **multilenguaje y multiplataforma**, que está pensada para correr sobre distintos entornos operativos, y que pretende unificar los distintos paradigmas de la programación (escritorio, cliente-servidor, WEB, APIs, Datos, Business Intelligence, etc.) en un mismo ambiente de trabajo.

El concepto de .NET es demasiado amplio, simplemente lo llamamos “**dotNET**” o “**punto NET**”, así que lo resumimos en pocas palabras:

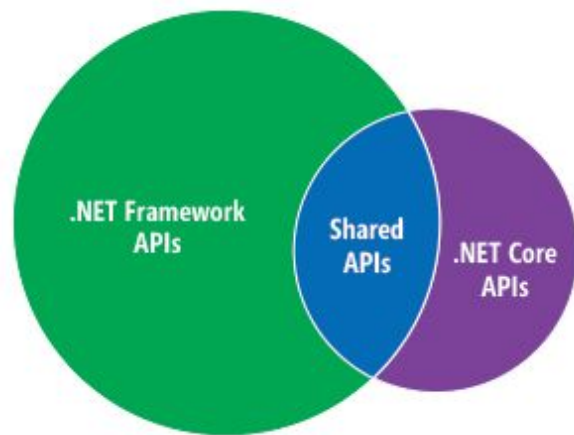
.NET está formado por cuatro componentes principales:

1. **El CLR (Common Language Runtime).**
Es el entorno de ejecución para las aplicaciones codificadas en alguno de los diferentes lenguajes que cumplen con los requisitos sintácticos y semánticos de .NET.
2. **El MSIL (Microsoft Intermediate Language).** Representa el *código objeto* al que es transformado el *código fuente* escrito en cualquiera de los lenguajes aceptados por .NET. Este código fuente luego va a ser compilado a código nativo por el compilador **JIT** (Just In Time) que provee el CLR en tiempo de ejecución.
3. **La biblioteca de clases .NET Framework.**
Es una extensísima colección de *clases base* (plantillas), para todos los propósitos imaginables, que es implementada y usada por todos los lenguajes e IDEs que corren sobre .NET.
4. **El entorno de programación (IDE).** Es el ambiente de trabajo propiamente dicho, conformado por diferentes herramientas y diversos asistentes que permiten que el programador pueda diseñar, codificar y testear sus aplicaciones. Existen varios (Sharp Develop, RAD Studio, etc.), pero en este curso usaremos el propio de Microsoft: *Visual Studio*.

Implementaciones de .NET

Orientándonos más a lo técnico y no tanto a los lenguajes de programación en que se puede programar, una aplicación de .NET se desarrolla y se ejecuta en una o varias *implementaciones de .NET*.

Las implementaciones de .NET incluyen estos componentes: **.NET Framework**, **.NET Core** y **Mono**. Además, hay una especificación de API (Application Programming Interface, que es un conjunto de librerías de código), común a todas las implementaciones de .NET que se denomina **.NET Standard**. Veremos una breve introducción a cada uno de estos conceptos.

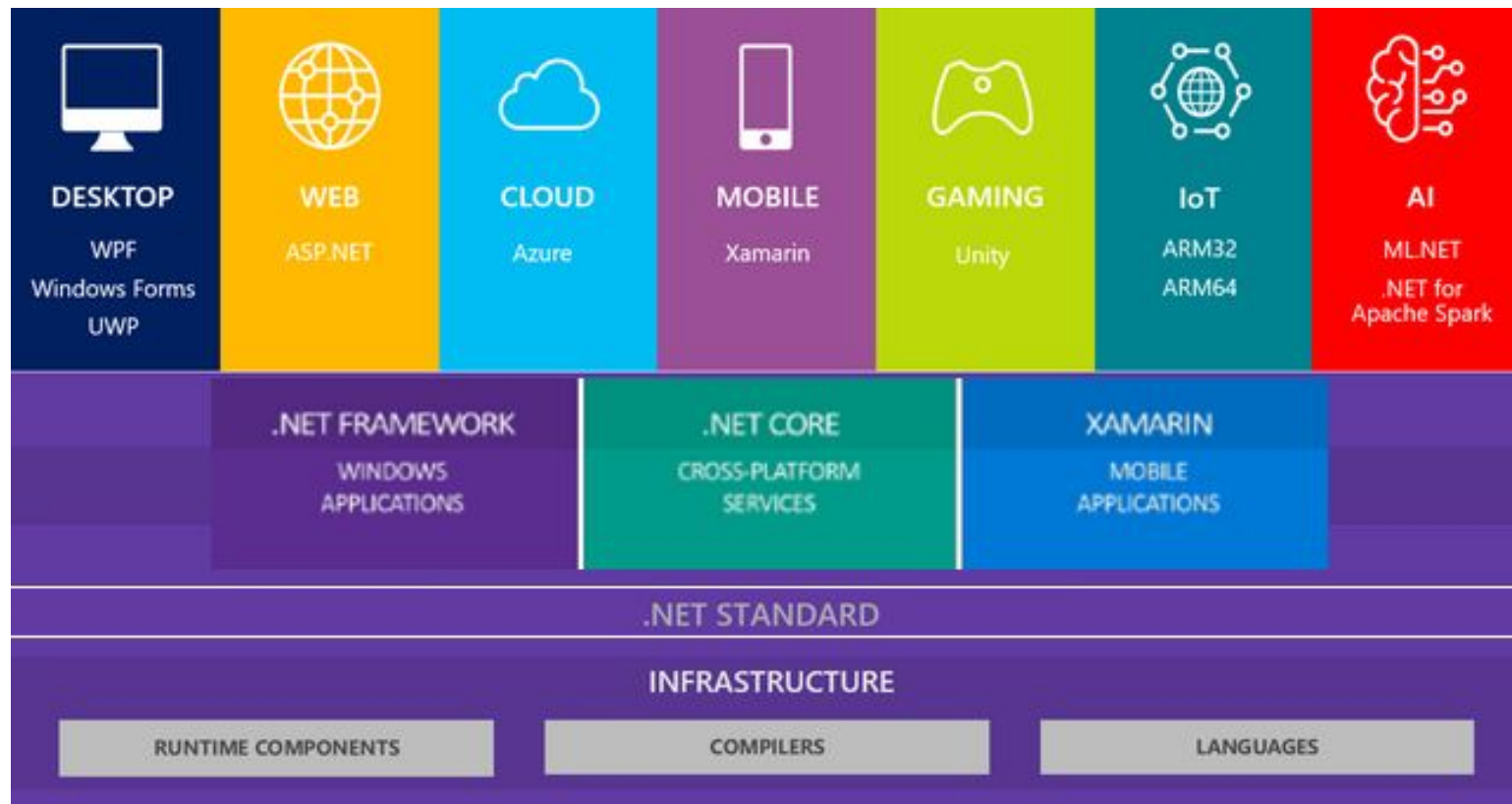


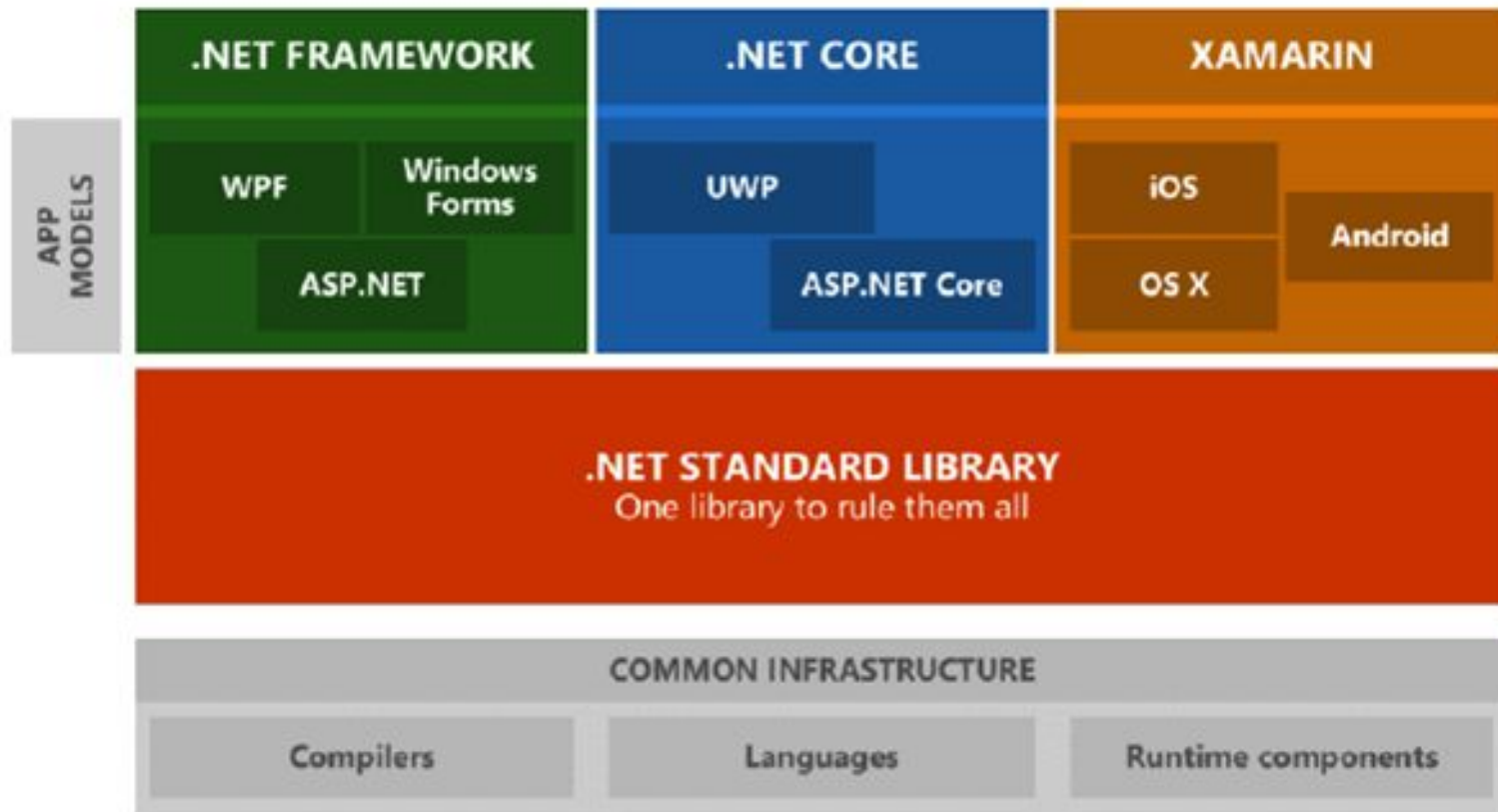
.NET Standard

.NET Standard es un conjunto de API que se implementa mediante la biblioteca de **clases base** de una implementación de .NET. Más formalmente, es una especificación de API de .NET que constituye un **conjunto uniforme de contratos** contra los que se compila el código. Estos contratos se aplican en cada implementación de .NET.

Esto permite la portabilidad entre diferentes implementaciones de .NET, de forma que el código se puede ejecutar en cualquier parte; por ejemplo, esto logra que la implementación .NET Core sea multiplataforma.

.NET Standard es también una [plataforma de destino](#). Si el código tiene como destino una versión de .NET Standard, se puede ejecutar en cualquier implementación de .NET que sea compatible con esa versión de .NET Standard.





Implementaciones de .NET

Cada implementación de .NET incluye los siguientes componentes, luego profundizaremos conceptualmente en algunos de ellos:

Uno o varios entornos de ejecución, por ejemplo:

- **CLR** para .NET Framework ([Common Language Runtime](#))
- **CoreCLR** y **CoreRT** para .NET Core ([.NET Core Common Language Runtime](#) y [.NET Core Runtime](#))

Una biblioteca de clases que implementa .NET Standard y puede implementar API adicionales, por ejemplo:

- Biblioteca de clases base de .NET Framework
- Biblioteca de clases base de .NET Core ([.NET Core Base Class Library](#))

Opcionalmente, uno o varios marcos de trabajo de la aplicación, por ejemplo:

- [ASP.NET](#) para .NET Framework y para .NET Core
- [Windows Forms](#) para .NET Framework
- [Windows Presentation Foundation \(WPF\)](#) para .NET Framework.

Opcionalmente, herramientas de desarrollo. Algunas herramientas de desarrollo se comparten entre varias implementaciones.

Hay cuatro implementaciones principales de .NET que Microsoft desarrolla y mantiene activamente: **.NET Core, .NET Framework, Mono y UWP.**



.NET Core

.NET Core es una implementación multiplataforma de .NET diseñada para aplicaciones instaladas tanto en servidores propios, como servidores en la nube a gran escala.

.NET Core puede ejecutarse en entornos Windows, macOS y Linux. Implementa .NET Standard, de forma que cualquier código que tenga como destino .NET Standard se pueda ejecutar en .NET Core.

ASP.NET Core se ejecuta en .NET Core.



.NET Framework

.NET Framework es la implementación original de .NET que existe desde 2002. Es el mismo entorno que los desarrolladores de .NET han usado siempre.

Las versiones 4.5 y posteriores implementan .NET Standard, de forma que el código que tiene como destino .NET Standard se puede ejecutar en esas versiones de .NET Framework.

Contiene API específicas de Windows adicionales, como la API para el desarrollo de aplicaciones de escritorio con Windows Forms y WPF. .NET Framework está optimizado para crear aplicaciones de escritorio de Windows.

¿Qué es C#?

C# (se pronuncia “se sharp”) es un lenguaje que pretende reunir lo mejor de los diversos lenguajes que compilan a nativo (C/C++, Object Pascal, ADA, etc.), y de aquellos interpretados (Java, Perl, etc.) en uno solo y que, además, pueda correr sobre diversos sistemas operativos. Al respecto, existe un proyecto paralelo al desarrollo de .NET de Microsoft, que se denomina “Mono” que transportó el lenguaje C# y gran parte del framework .NET a Unix/Linux/Solaris.

C# toma gran parte de la sintaxis de C/C++, y muchas de las características de Eiffel (un lenguaje de laboratorio que ideó Bertrand Meyer), poniendo a disposición del desarrollador un potente lenguaje 100% orientado a objetos. Si bien el CLR, acepta muchos otros lenguajes, el framework .NET está 100% escrito en C#; por lo tanto, ***es el lenguaje base para .Net.***



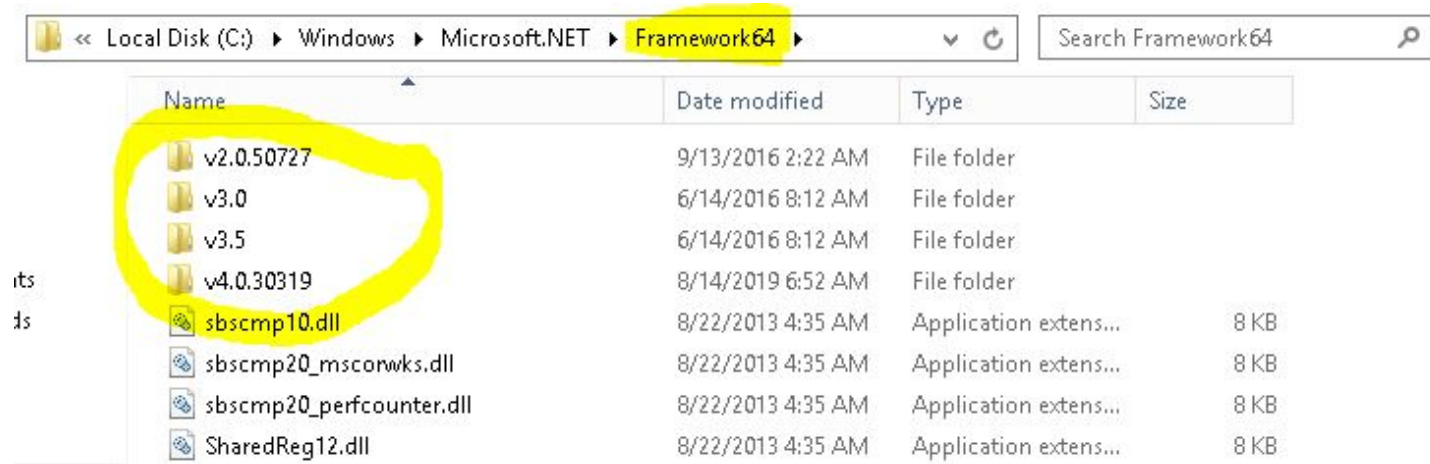
Entorno de desarrollo

El compilador

Cuando se instala en cada computadora el SDK de Microsoft .NET, se crea el directorio `C:\WINDOWS\Microsoft.NET\Framework\v2.0.50727\..` , dentro del cual se alojan los compiladores de línea.

En nuestro caso, el compilador para C# es `"csc.exe"` que puede ser invocado desde la línea de comandos (Inicio | Ejecutar | cmd). Cuando se invoca al compilador pasándole como parámetro un nombre de archivo cualquiera con la extensión `".cs"`, convierte a este en un archivo `*.exe` (ejecutable), pero este NO es un ejecutable NATIVO (es decir autónomo), sino que necesita del framework .NET con el CLR instalado en la misma máquina, para poder correr.

Ejemplo:



File Explorer window showing the path: Local Disk (C:) > Windows > Microsoft.NET > Framework64. The search bar contains "Search Framework64".

Name	Date modified	Type	Size
v2.0.50727	9/13/2016 2:22 AM	File folder	
v3.0	6/14/2016 8:12 AM	File folder	
v3.5	6/14/2016 8:12 AM	File folder	
v4.0.30319	8/14/2019 6:52 AM	File folder	
sbscmp10.dll	8/22/2013 4:35 AM	Application extens...	8 KB
sbscmp20_mscomwks.dll	8/22/2013 4:35 AM	Application extens...	8 KB
sbscmp20_perfcounter.dll	8/22/2013 4:35 AM	Application extens...	8 KB
SharedReg12.dll	8/22/2013 4:35 AM	Application extens...	8 KB

Los archivos fuente (“*.cs”)

Los archivos de “código fuente” no son más que archivos de texto plano, que contienen las líneas del programa en C#, con las instrucciones que cada programador quiere que la aplicación lleve a cabo. Deben cumplir con las normas sintácticas del lenguaje, caso contrario, el compilador detecta los errores y no genera el ejecutable.

Veamos un primer ejemplo de aplicación de consola:



```

/*
Nombre del archivo: Saludo.cs
Autor: ...
Objetivo del programa: Mostrar por consola un mensaje de bienvenida
*/

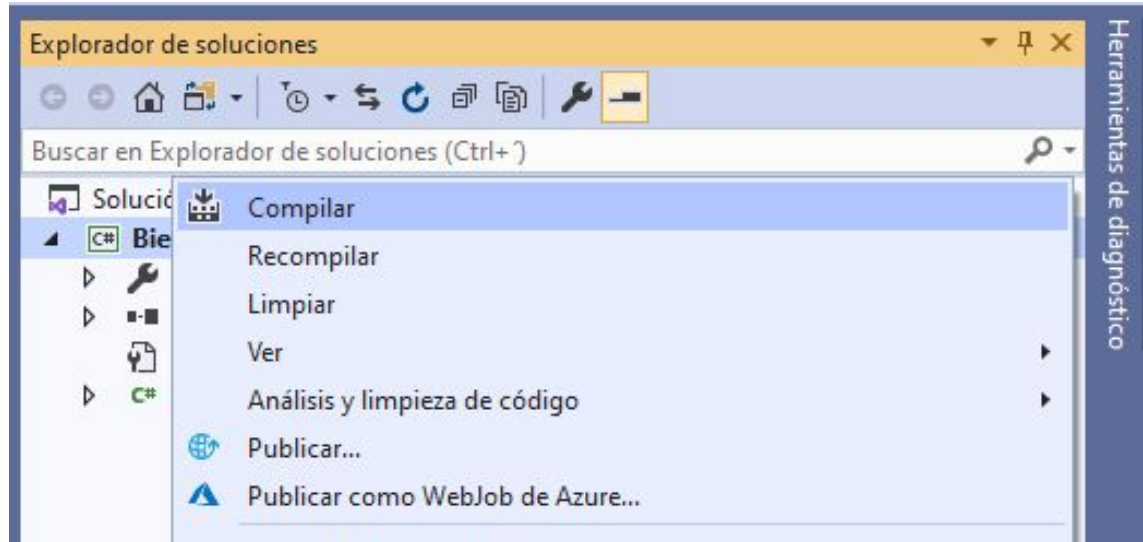
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

// Nombre del archivo ejecutable (mismo nombre que el Proyecto) .exe que se generará
namespace Bienvenida
{
    // Clase inicial del programa Saludo.cs
    class Saludo
    {
        // Punto de entrada a la aplicación (el código dentro de Main es lo primero en ejecutarse)
        static void Main()
        {
            // Muestra por pantalla el contenido entre "" dentro del paréntesis
            Console.WriteLine("Hola a todos !!! Bienvenidos a .Net !!!");

            // Espera que el usuario presione una tecla para continuar la ejecución del programa
            Console.ReadKey();
        }
    }
}

```

En la imagen anterior, codificamos el programa en el IDE de Visual Studio y aquí también podríamos compilarlo;



¿Cómo está formado el framework?

La arquitectura del framework .NET, es la implementación del CLI (*Common Language Runtime*) por parte de Microsoft para los lenguajes C#, Visual Basic, J#, ASP y Javascript, más varios paquetes que dan soporte a interfaces de usuario, acceso a datos, XML y WEB agrupadas en una *Librería de Clase Base* (BCL) que está formada por cientos de tipos de datos.

Esta librería está escrita en MSIL, por lo que puede usarse desde cualquier lenguaje cuyo compilador genere MSIL. A través de las clases suministradas en ella es posible desarrollar cualquier tipo de aplicación, desde las tradicionales aplicaciones de ventanas, consola o servicio de Windows NT hasta los novedosos servicios Web y páginas ASP.NET.

Dada su amplitud ha sido necesario organizar las clases en ella incluida en **espacios de nombres** que agrupen clases con funcionalidades similares, es una clasificación. Por ejemplo, los espacios de nombres más usados son:

Espacio de nombres	Utilidad de los tipos de datos que contiene
System	Tipos muy frecuentemente usados, como los tipos básicos, tablas, excepciones, fechas, números aleatorios, recolector de basura, entrada/salida en consola, etc.
System.Collections	Colecciones de datos de uso común como pilas, colas, listas, diccionarios, etc.
System.Data	Manipulación de bases de datos. Forman la denominada arquitectura ADO.NET.
System.IO	Manipulación de archivos y otros flujos de datos.
System.Net	Realización de comunicaciones en red.
System.Reflection	Acceso a los metadatos que acompañan a los módulos de código.
System.Runtime.Remoting	Acceso a objetos remotos.
System.Security	Acceso a la política de seguridad en que se basa el CLR.
System.Threading	Manipulación de hilos de ejecución.
System.Web.UI.WebControls	Creación de interfaces de usuario basadas en ventanas para aplicaciones Web.
System.Windows.Forms	Creación de interfaces de usuario basadas en ventanas para aplicaciones estándar.
System.XML	Acceso a datos en formato XML.

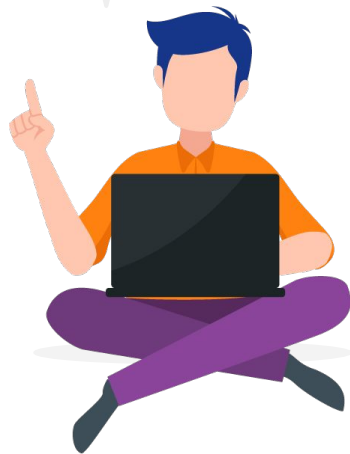
Comentarios

¿Qué es un comentario?

Un **comentario** es texto que se incluye en el código fuente para facilitar su lectura a los programadores y cuyo contenido es, por defecto, completamente ignorado por el compilador.

En C# hay dos formas de escribir comentarios. La primera consiste en encerrar todo el texto que se desee comentar entre caracteres `/*` y `*/` siguiendo la siguiente sintaxis: `/*<texto>*/`

```
/*<esto es un comentario>*/
```



Utilización de comentarios

Estos comentarios pueden abarcar tantas líneas como sea necesario. Por ejemplo:

```
/*  
Esto es un comentario  
que muestra cómo se comentan varias líneas  
*/
```

No es posible anidar comentarios de este tipo. Es decir, no vale escribir comentarios como el siguiente:

```
/* Comentario contenedor /* Comentario contenido */ */
```


Dado que muchas veces los comentarios que se escriben son muy cortos y no suelen ocupar más de una línea, C# ofrece una sintaxis alternativa más compacta para la escritura este tipo de comentarios en las que se considera como indicador del comienzo del comentario la pareja de caracteres **//** y como indicador de su final el fin de línea. Por tanto, la sintaxis que siguen estos comentarios es:

// <texto>

Y un ejemplo de su uso es:

```
// Este es un comentario de una sola línea
```

Estos comentarios de una sola línea sí que pueden anidarse sin ningún problema. Por ejemplo, el siguiente comentario es perfectamente válido:

```
// Comentario contenedor // Comentario contenido
```

El IDE de Visual Studio, tiene en botones dos funcionalidades: una para comentar un bloque de instrucciones y otra para “descomentarlos”, resultando en muchas ocasiones más ágiles.



¡Muchas gracias!

¡Sigamos trabajando!